

Aula 4 Parte 1

Caracterização e Preparação dos Dados

Data	Segunda-feira, 24/08/2026 sessão 7 de 36
Unidade da ementa	3. Processamento de dados: caracterização e preparação
Local	Laboratório de Informática (cada aluno em uma máquina)
Arquivos	lab04_parte1_demo.py • lab04_parte1_exercicios.py
Entrega	lab04_parte1_exercicios.py preenchido e executado + relatório de conclusão

Objetivos desta sessão

1. Realizar uma primeira caracterização de um dataset: dimensões, tipos de dados, valores ausentes e distribuições.
2. Identificar os problemas de qualidade mais comuns: valores ausentes, outliers e atributos categóricos sem codificação.
3. Diferenciar as transformações básicas de preparação: imputação, codificação de categorias e normalização/padronização.
4. Compreender o conceito de vazamento de dados (data leakage) e a regra de ajustar no treino, aplicar no teste.
5. Observar, na prática, como um ColumnTransformer organiza essas etapas em um único objeto.

1. Retomada: de datasets prontos para dados crus

Nas três primeiras semanas você trabalhou com datasets do scikit-learn Iris e Wine já limpos, sem valores ausentes e com todas as colunas numéricas. Isso não é um acaso: são datasets didáticos, preparados exatamente para que a aula pudesse focar no algoritmo. Fora da sala de aula, os dados quase nunca chegam assim. Hoje muda o ponto de partida: como transformar dados "crus" do jeito que chegam do mundo real em um X que um algoritmo consegue efetivamente usar?

2. Por que caracterizar antes de modelar

Antes de escolher um algoritmo, é preciso olhar para os dados. Uma caracterização inicial muitas vezes chamada de EDA (Exploratory Data Analysis, ou análise exploratória de dados) responde perguntas simples, mas decisivas: quantas linhas e colunas existem? Qual o tipo de cada coluna (numérica ou categórica)? Existem valores ausentes, e em quais colunas? Os valores fazem sentido, ou há algo muito fora do esperado?

Nenhum algoritmo, por mais sofisticado que seja, resolve sozinho um problema que já está nos dados de entrada. Pular essa etapa é a causa mais comum de um modelo que "não funciona" quando, na verdade, o problema nunca esteve no algoritmo.

3. Problemas comuns de qualidade dos dados

3.1 Valores ausentes

Valores ausentes (missing values) aparecem por diversos motivos: um campo opcional que o usuário não preencheu, uma falha no sensor que coletou o dado, um registro que não se aplica a determinado caso. Em pandas, aparecem como NaN e são contados com `df.isna().sum()`. Duas estratégias básicas: descartar (a linha ou a coluna inteira, quando poucos dados são afetados) ou imputar (substituir por um valor estimado média, mediana ou moda são as mais comuns). Descartar é simples, mas custa dados; imputar preserva o tamanho do dataset, ao custo de introduzir uma estimativa no lugar do valor real.

3.2 Outliers

Outliers são valores muito distantes do padrão observado no restante dos dados. Podem ser erro de digitação (idade = 300) ou um caso real, porém atípico (um cliente com renda muito acima da média). Uma forma simples e amplamente usada de sinalizá-los é a regra do IQR (intervalo interquartil): calcula-se Q1 e Q3 (primeiro e terceiro quartis), o $IQR = Q3 - Q1$, e marca-se como outlier tudo o que fica acima de $Q3 + 1,5 \times IQR$ (ou abaixo de $Q1 - 1,5 \times IQR$). A regra aponta onde olhar não decide sozinha se o valor deve ser removido, corrigido ou mantido.

3.3 Atributos categóricos

Algoritmos baseados em distância ou em álgebra linear K-NN, K-Means, regressão linear, entre outros trabalham com números, não com texto. Uma coluna como cidade ("SP", "RJ", "MG") precisa ser codificada. A codificação one-hot cria uma coluna binária (0 ou 1) para cada categoria, sem introduzir nenhuma relação de ordem entre elas.

O erro conceitual mais comum

Usar códigos numéricos arbitrários (SP=0, RJ=1, MG=2) como se fossem uma escala. Isso cria uma relação de ordem e de distância que não existe de fato entre as categorias, e o algoritmo passa a "enxergar" essa falsa relação como se "MG" fosse "maior" ou "mais distante" de "SP" do que "RJ" é.

3.4 Escalas diferentes

Retomando a lição das Aulas 2 e 3: quando os atributos numéricos estão em escalas muito diferentes (idade em dezenas, renda em milhares), a distância euclidiana passa a ser dominada pela coluna de maior amplitude. A padronização (StandardScaler, que centraliza em média 0 e desvio-padrão 1) ou a normalização (Min-Max, que reescala para um intervalo fixo, geralmente [0, 1]) resolvem esse problema o mesmo remédio que já apareceu para K-NN e K-Means.

4. Vazamento de dados (data leakage)

Vazamento de dados acontece quando informação do conjunto de teste ou do futuro, em problemas com tempo influencia, mesmo que indiretamente, o treinamento do modelo. Isso infla artificialmente as métricas de avaliação e engana sobre a capacidade real de generalização.

A regra de ouro: qualquer transformação que "aprende" algo dos dados a média usada para imputar, o mínimo e o máximo usados para normalizar, as categorias vistas pelo one-hot deve ser ajustada (fit) somente no conjunto de treino, e depois aplicada (transform) tanto ao treino quanto ao teste. É exatamente isso que um Pipeline combinado com um ColumnTransformer faz de forma organizada, quando encaixado corretamente depois do `train_test_split`.

5. Demonstração guiada: caracterizando e preparando um dataset

Trabalhamos com um dataset sintético de solicitações de empréstimo (450 linhas), construído com valores ausentes em três colunas, uma coluna categórica (cidade) e outliers propositalmente em renda para reproduzir, de forma controlada, os problemas descritos acima.

```
import numpy as np
import pandas as pd

df = gerar_dados_emprestimo(n=450, semente=42)
print(df.shape)          # (450, 6)
print(df.dtypes)         # idade, renda, tempo_emprego, score_credito: float64
                        # cidade: object    |    aprovado: int64
print(df.isna().sum())   # idade: 41  renda: 42  cidade: 33
```

Trecho 1 shape, dtypes e isna().sum() são os três primeiros comandos de qualquer caracterização: quantos dados existem, de que tipo, e onde faltam valores.

```
q1, q3 = df["renda"].quantile([0.25, 0.75])
iqr = q3 - q1
limite_superior = q3 + 1.5 * iqr
outliers = df["renda"] > limite_superior
print(outliers.sum())    # 20 outliers de 450 linhas
```

Trecho 2 a regra do IQR aplicada a renda encontra 20 outliers (4,4% das linhas), coerente com o fato de termos inserido alguns valores propositalmente muito altos na construção do dataset.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

colunas_num = ["idade", "renda", "tempo_emprego", "score_credito"]
colunas_cat = ["cidade"]

prep = ColumnTransformer([
    ("num", Pipeline([("imputer", SimpleImputer(strategy="median")),
                     ("scaler", StandardScaler())])), colunas_num),
    ("cat", Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                     ("onehot", OneHotEncoder(handle_unknown="ignore"))])), colunas_cat),
])

X = df.drop(columns="aprovado")
X_pronto = prep.fit_transform(X)
print(X.shape, "->", X_pronto.shape)  # (450, 5) -> (450, 8)
```

Trecho 3 o ColumnTransformer aplica um Pipeline diferente a cada grupo de colunas: imputação por mediana e padronização nas numéricas, imputação por moda e one-hot na categórica. O resultado não tem mais nenhum NaN, e cidade virou quatro colunas de 0/1 (uma por categoria), o que explica o shape passar de 5 para 8 colunas.

Um cuidado importante

No Trecho 3 usamos fit_transform no X inteiro, sem separar treino e teste um atalho didático só para observar a transformação acontecendo. Fazer isso "para valer", em um projeto real, é a causa mais comum de vazamento de dados: a mediana e as categorias aprendidas passam a "conhecer" também o que deveria estar reservado para o teste. O jeito correto dividir primeiro, ajustar só no treino é o assunto central da Parte 2 (25/08).

O que guardar desta sessão

- Caracterizar antes de modelar: `shape`, `dtypes`, `isna().sum()` e `describe()` são o primeiro passo, sempre.
- Valores ausentes: descartar custa dados; imputar preserva o tamanho do dataset ao custo de uma estimativa.
- Outliers: a regra do IQR aponta onde olhar, não decide sozinha o que fazer com cada caso.
- Categóricos exigem codificação one-hot evita criar uma relação de ordem que não existe.
- Vazamento de dados: qualquer transformação ajustada (fit) deve olhar só para o treino, nunca para o teste.